

Анатомия аргументов: значения по умолчанию и современная типизация в Python

1. Параметры vs Аргументы

Коротко напомним разницу, так как это важно для понимания документации:

- **Параметры** - это переменные в скобках при определении функции (def).
- **Аргументы** - это конкретные данные, которые вы передаете в функцию при её вызове.

2. Значения по умолчанию (Default Arguments)

Иногда нам нужно, чтобы параметр имел какое-то значение «по умолчанию», если пользователь решил его не указывать.

Правило: Параметры со значениями по умолчанию всегда должны стоять после обязательных параметров.

Шаблон:

```
def имя_функции(обязательный, необязательный="значение"):  
    pass
```

Реальный пример:

```
def send_message(text, user="Гость"):  
    print(f"Сообщение для {user}: {text}")  
  
send_message("Привет!")           # Использует "Гость"  
send_message("Привет!", "Алихан") # Перезапишет "Гость"  
на "Алихан"
```

3. Аннотации типов (Type Hinting)

В современном Python (начиная с 3.5+) принято указывать типы аргументов и тип возвращаемого значения. Это не заставляет Python выдавать ошибку, если тип не совпал, но помогает программисту и редактору кода (VS Code, PyCharm) видеть ошибки заранее.

Синтаксис:

- Тип аргумента указывается через двоеточие: аргумент: тип.
- Тип возвращаемого значения указывается через стрелочку: -> тип.

Шаблон:

```
def calc_sum(a: int, b: int) -> int:
    return a + b
```

4. Какое значение должна отдавать функция?

Функция может возвращать любой тип данных (int, str, list, dict и т.д.). Важно, чтобы это было предсказуемо.

- Если функция выполняет математику - возвращайте число.
- Если функция ищет что-то - возвращайте объект или None, если ничего не найдено.
- Если функция проверяет условие - возвращайте bool (True/False).

Пример со всеми фишками:

Представьте функцию для расчета стоимости доставки в онлайн-магазине.

```
def calculate_delivery(price: float, country: str =
"Казахстан") -> float:
    """Рассчитывает стоимость заказа с учетом доставки."""
    if country == "Казахстан":
        delivery_fee = 0.0
    else:
        delivery_fee = 5000.0

    return price + delivery_fee

# Вызов функции
final_price = calculate_delivery(15000.50,
country="США")
print(f"Итого к оплате: {final_price}")
```

Реальные примеры применения

1. **Логирование:** Функция `log(message, level="INFO")`. Большинство логов - это просто информация, но иногда мы можем передать "ERROR".
2. **Работа с текстом:** Функция `format_string(text: str, uppercase: bool = False) -> str`. По умолчанию возвращает обычный текст, но может и капсом.
3. **API и Базы данных:** Параметры пагинации, например `get_users(limit: int = 10)`. По умолчанию берем 10 записей.

Задачи для закрепления

Задача 1: «Умный калькулятор»

Напишите функцию `power(base: int, exp: int = 2) -> int`. Она должна возводить число `base` в степень `exp`. По умолчанию функция должна возвращать **квадрат** числа. Укажите типы данных.

Задача 2: «Форматирование валюты»

Создайте функцию `format_price(amount: float, currency: str = "тг") -> str`. Она должна возвращать строку вида `"1500.00 тг"`. Убедитесь, что цена выводится с двумя знаками после запятой.

Задача 3: «Валидатор пароля»

Напишите функцию `is_valid_password(password: str, min_length: int = 8) -> bool`. Она должна возвращать `True`, если длина пароля больше или равна `min_length`, иначе `False`.

Задача 4: «Приветствие с должностью»

Напишите функцию `greet_employee(name: str, position: str = "стажер") -> str`. Она должна возвращать строку: `"Сотрудник [имя], должность: [должность]"`. Попробуйте вызвать её, передав только имя.

Задача 5: «Список покупок»

Создайте функцию `add_to_cart(item_name: str, quantity: int = 1) -> dict`. Она должна возвращать словарь вида `{"item": "название", "count": количество}`. Вызовите функцию для одного яблока (не передавая количество) и для 5 апельсинов.



Короткий вывод:

- **Значения по умолчанию** избавляют от необходимости передавать одни и те же данные при каждом вызове.
- **Типизация** делает ваш код понятным для других разработчиков и ИИ-помощников.
- **Стрелочка ->** в объявлении функции четко говорит: «Вот что вы получите на выходе».